TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG

BÁO CÁO ĐỒ ÁN MÔN HỌC VI DIỀU KHIỂN

Đề tài:

Thiết Bị Đồng Hồ Matrix Thời Gian Thực Đa Tính Năng Trên Vi Điều Khiển Stm32

Giảng viên: PGS.TS Lê Đức Hùng

Nhóm:7

Họ và Tên	***MSSV***
Hồ Trọng Hải	23207050
Phạm Văn Phúc	23207096
Lê Trần Viết Thịnh	23207111
Nguyễn Trần Quốc Bảo	23207038

Thành phố Hồ Chí Minh, ngày 31 tháng 3 năm 2025

BẢNG THỐNG KÊ CẬP NHẬT TÀI LIỆU

Phiên bản	Mô tả	Thành viên	Ngày	Xác nhận	Ngày xác nhận
1.0	Tạo bài báo cáo	Nguyễn Trần Quốc Bảo	31/03/2025	Phạm Văn Phúc	18/04/2025
2.0	Bổ sung chương I	Nguyễn Trần Quốc Bảo	24/02/2025	Phạm Văn Phúc	18/04/2025
2.1	Bổ sung chương II	Hồ Trọng Hải	26/03/2025	Phạm Văn Phúc	18/04/2025
3.0	Chỉnh sửa chương I và II	Phạm Văn Phúc	31/03/2025	Phạm Văn Phúc	18/04/2025
4.0	Chỉnh sửa bố cục, tạo mục lục	Lê Trần Viết Thịnh	31/03/2025	Phạm Văn Phúc	18/04/2025
4.1	Chỉnh sửa nội dung chương I	Nguyễn Trần Quốc Bảo	02/04/2025	Phạm Văn Phúc	18/04/2025
4.2	Chỉnh sửa nội dung chương II	Hồ Trọng Hải	05/04/2025	Phạm Văn Phúc	18/04/2025
4.3	Tạo phụ lục, bổ sung phân công	Lê Trần Viết Thịnh	10/04/2025	Phạm Văn Phúc	18/04/2025
5.0	Kiểm tra, sửa lỗi Hoàn thiện	Phạm Văn Phúc	18/04/2025	Phạm Văn Phúc	18/04/2025

BÁO CÁO KĨ NĂNG

1. Kế hoạch thực hiện

Nhiệm vụ	Người thực hiện	Ngày bắt đầu	Ngày kết thúc	Tiến trình
Họp thống nhất ý tưởng	Cả nhóm	14/02/2025	14/02/2025	Hoàn thành
Khởi tạo báo cáo và thuyết trình	Nguyễn Trần Quốc Bảo Hồ Trọng Hải	15/02/1025	15/02/2025	Hoàn thành
Tìm kiếm, mua linh kiện	Phạm Văn Phúc Hồ Trọng Hải	14/02/2025	14/02/2025	Hoàn thành
Nghiên cứu DS3231	Phạm Văn Phúc	01/03/2025	10/03/2025	Hoàn thành
Thiết kế mạch trên Fritzing	Hồ Trọng Hải	30/03/2025	31/03/2025	Hoàn thành
Lắp ráp, chạy thử lần 1	Cả nhóm	31/03/2025	31/03/2025	Hoàn thành
Sửa chửa, chạy thử lần 2	Phạm Văn Phúc	31/03/2025	02/04/2025	Hoàn thành
Bổ sung, chỉnh sửa báo cáo	Nguyễn Trần Quốc Bảo	31/03/2025	02/04/2025	Hoàn thành
Bổ sung, chỉnh sửa thuyết trinh	Lê Trần Viết Thịnh	15/04/2025	17/04/2025	Hoàn thành
Chạy thử lần 3, hoàn thiện	Cả nhóm	15/04/2025	17/04/2025	Hoàn thành
Kiểm tra, sửa chữa	Cả nhóm	17/04/2025	18/04/2025	Hoàn thành
Báo cáo	Cả nhóm	21/04/2025	21/04/2026	Hoàn thành

2. Tự đánh giá

Kĩ năng	Đánh giá (*)
Làm việc nhóm, giao tiếp, hành xử chuyên nghiệp	**A**
Làm việc độc lập	**A**
Kĩ năng thuyết trình	**A**
Giao tiếp kĩ thuật	**B**
Tư duy sáng tạo	**A**
Quản lí dự án/thời gian thực h iện dự án	**A**
Hình thành nội dung, xác định và giải quyết vấn đề	**A**
Kiến thức có được thông qua dự án	**A**

MỤC LỤC

LỜI MỞ ĐẦU **1**

CHƯƠNG I : GIỚI THIỆU SƠ LƯỢC LINH KIỆN **3**

1. KIT STM32F103C8T6 BluePill ***3***

1.1. Đôi nét về KIT STM32F103C8T6 BluePill **3**

1.2. Thông số kĩ thuật STM32F103C8T6 ..... **3**

1.3. Danh sách chân **4**

2. Các thiết bị đầu vào ***4***

2.1. Module RealTime Clock DS3231 ..... **4**

2.2. Module Touch Sensor TTP223 ..... **5**

3. Các thiết bị đầu ra ***6***

3.1. LED Matrix7219_8x32 ..... **6**

3.2. Buzzer 5VDC **6**

CHƯƠNG II: THIẾT KẾ, CẤU TẠO VÀ NGUYÊN LÍ HOẠT ĐỘNG **8**

1.1. Sơ đồ thiết kế **8**

1.2. Hiện thực sản phẩm trên breadboard **9**

1.3. Hoàn thiện sản phẩm **9**

2. Chương trình nạp ***10***

2.1. Thiết lập thư viện **10**

2.2. Thư viện dùng cho LED Matrix7219_8x32 ..... **11**

2.3. Tạo Struct và định nghĩa hàm cho DS3231 ..... **16**

2.4. Tạo hàm dùng cho 3 nút nhấn PB3, PB4, PB5 (tương ứng 3 MODE 1, 2, 3) **21**

2.5. **Khai báo biến và khởi tạo các init** **22**

2.6. **MODE1: Hiển thị thời gian, MODE2: Hiển thị nhiệt độ, MODE3: Bộ đếm (Sau 45p sẽ báo thức)** **23**

3. Nguyên lí hoạt động theo Block Chart ***25***

KẾT LUẬN **27**

TÀI LIỆU THAM KHẢO ..... Error! Bookmark not defined.

DANH MỤC HÌNH

Hình 1: KIT STM32F103C8T6 BluePill 3
Hình 2: Module RTC DS3231 4
Hình 3: Module Touch Sensor TTP223 5
Hình 4: Led MaTrix7219 8x32 6
Hình 5: Buzzer 5VDC 7
Hình 6: Thiết kế mạch trên Fritzing 8
Hình 7: Lắp ráp mạch trên breadboard và chạy thử 9
Hình 8: Hàn chân và đóng hộp sản phẩm 9

DANH MỤC BẢNG

Bảng 1: Danh sách chân STM32F103C8T6 ..... *4*

Bảng 2: Danh sách nối chân linh kiện *8,9*

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ phát triển không ngừng, các thiết bị điện tử thông minh ngày càng đóng vai trò quan trọng trong đời sống hiện đại. Đặc biệt, các hệ thống nhúng và vi điều khiển như STM32 đang trở thành nền tảng chủ chốt trong việc phát triển các sản phẩm điện tử linh hoạt, tiết kiệm năng lượng và hiệu suất cao.

Đề tài "Xây Dựng Ý Tưởng Cho Thiết Bị Đồng Hồ Matrix Thời Gian Thực Đa Tính Năng Trên Vi Điều Khiển STM32" được lựa chọn với mục tiêu nghiên cứu và phát triển một thiết bị đồng hồ thông minh có khả năng hiển thị thời gian theo dạng ma trận, đồng thời tích hợp các tính năng thông minh và hữu ích trong cuộc sống hàng ngày. Lý do chọn đề tài này xuất phát từ sự phát triển nhanh chóng của công nghệ điện tử và vi điều khiển, đặc biệt là các vi điều khiển STM32, nổi bật với khả năng xử lý mạnh mẽ và tiết kiệm năng lượng. Bên cạnh đó, việc sử dụng màn hình ma trận để hiển thị thời gian không chỉ mang lại sự trực quan, mà còn giúp khám phá khả năng tạo ra những giao diện người dùng linh hoạt và hiện đại.

Một trong những lý do hình thành ý tưởng cho đề tài này đến từ thực trạng sinh viên dễ bị mất tập trung trong quá trình học tập khi phải sử dụng điện thoại để xem giờ. Việc kiểm tra thời gian trên điện thoại thường dẫn đến sự xao nhãng do vô tình bị thu hút bởi các ứng dụng khác như mạng xã hội, trò chơi hoặc thông báo ngoài ý muốn. Do đó, nhóm quyết định phát triển một thiết bị đồng hồ độc lập, hiển thị thời gian trên màn hình LED ma trận theo thời gian thực, loại bỏ nhu cầu phải sử dụng điện thoại và giúp người dùng tập trung hơn vào việc học.

Ý tưởng cho đề tài "Xây Dựng Ý Tưởng Cho Thiết Bị Đồng Hồ Matrix Thời Gian Thực Đa Tính Năng Trên Vi Điều Khiển STM32" cũng được khơi nguồn từ nội dung thực hành Lab 9 về giao thức I2C, kết hợp với kiến thức lý thuyết về các chuẩn giao tiếp SPI và I2C đã học trong tuần 7 của môn Vi Điều Khiển. Đồng thời, đề tài cũng chịu ảnh hưởng từ sự phát triển của đồng hồ thông minh hiện đại, vốn không chỉ giới hạn ở chức năng hiển thị thời gian mà còn tích hợp nhiều tính năng tiên tiến như theo dõi sức khỏe, đo nhiệt độ môi trường và điều khiển từ xa các thiết bị điện tử. Việc lựa chọn công nghệ màn hình LED ma trận và vi điều khiển STM32 xuất phát từ tính linh hoạt và khả năng đáp ứng hiệu quả các yêu cầu về đa chức năng trong một thiết bị có kích thước nhỏ gọn.

Trong thực tế, ý tưởng này đã được phát triển và ứng dụng tại nhiều nơi. Dựa vào ý tưởng này để nhóm chúng em có thể hiểu hơn về cách hoạt động của vi điều khiển STM32F103, LED Matrix7219 và mạch thời gian thực RTC DS3221. Từ đó những kiến thức thu được sẽ được đưa vào thực tiễn giúp chúng em nắm bắt được kiến thức thực tế, vận dụng và phát triển tương lai.

Tầm nhìn tương lai của ý tưởng này là phát triển một thiết bị đồng hồ không chỉ là công cụ đo thời gian mà còn trở thành một trung tâm điều khiển đa chức năng. Với sự phát triển không ngừng của công nghệ IoT (Internet of Things) và các hệ sinh thái kết nối, thiết bị này có thể mở rộng để tích hợp các tính năng như điều khiển thông minh, kết nối với các thiết bị gia đình thông minh, cũng như khả năng nâng cấp phần mềm để hỗ trợ thêm các ứng dụng mới trong tương lai.

Tên đồ án: Thiết Bị Đồng Hồ Matrix Thời Gian Thực Đa Tính Năng trên vi điều khiển STM32F103

Thời gian thực hiện: 18/02/2025 – 18/04/2025

Nhóm: 7

Nhóm trưởng: Hồ Trọng Hải

Thành viên: Nguyễn Trần Quốc Bảo

Phạm Văn Phúc

Lê Trần Viết Thịnh

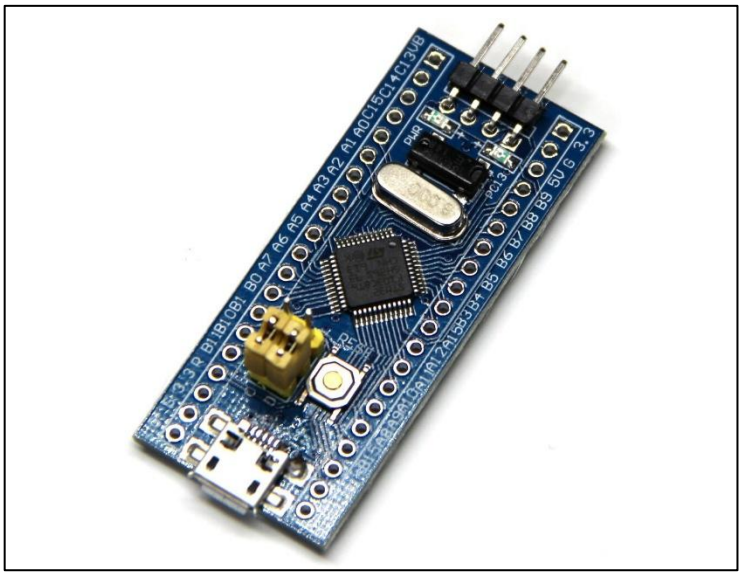
Lớp: 23DTV_CLC1

CHƯƠNG I : GIỚI THIỆU SƠ LƯỢC LINH KIỆN

1. KIT STM32F103C8T6 BluePill

1.1. Đôi nét về KIT STM32F103C8T6 BluePill

KIT BluePill sử dụng chip STM32F103C8T6 là một dòng chip phổ thông nhất của STM, có đầy đủ các ngoại vi cơ bản, bộ nhớ vừa đủ và giá thành rẻ, rất phù hợp với những người mới bắt đầu.



Hình 1: KIT STM32F103C8T6 BluePill

Đặc điểm nổi bật của ***KIT BluePill***:

- Tích hợp sẵn thạch anh ngoại 8MHz.
- Tích hợp cổng USB để nạp Bootloader hoặc sử dụng ngoại vi USB.
- Sử dụng chuẩn mạch nạp: SWD cho các loại mạch nạp ST-link, J-link.

STM32F103C8T6 thuộc họ F1 với lõi là ARM CORTEX M3. Đây là vi điều khiển 32 bit, tốc độ tối đa là 72Mhz. Giá thành cũng khá rẻ so với các loại vi điều khiển có chức năng tương tự. Mạch nạp cũng như công cụ lập trình khá đa dạng và dễ sử dụng.

1.2. Thông số kĩ thuật STM32F103C8T6

- ***Kiến trúc vi mạch:*** RISC
- ***Số chân:*** 47 chân
- ***Bộ nhớ:*** Bộ nhớ flash: 64 kB, SRAM: 20 kB
- ***Điện áp hoạt động:*** 2 – 3.6V
- ***Bộ tạo dao động thạch anh:*** 8 – 72 MHz
- ***Nhiệt độ hoạt động:*** -40 °C - 105 °C
- ***CPU clock:*** Phase-locked loop
- ***ADC:*** 2x(9 kênh 12-bit)
- ***DMA:*** 7 kênh
- ***Debug mode:*** Serial wire debug (SWD) và JTAG
- ***Timers:*** 7 bộ
- ***Kênh giao tiếp:*** 2 bộ I2C, 3 bộ USART, 2 SPI, 1 CAN, USB 2.0 full-speed interface

1.3. Danh sách chân

Bảng 1: Danh sách chân STM32F103C8T6

Loại chân	Tên chân
Power	3V3, 5V, GND
Analog	PA0-PA7, PB0-PB1
I/O	PA0-PA15, PB0-PB15, PC13-PC15
Ngắt ngoài	PA0-PA15, PB0-PB15, PC13-PC15
PWM	PA0-PA3, PA6-PA10, PB0-PB1, PB6-PB9
UART	TX1, RX1, TX2, RX2, TX3, RX3
SPI	MISO0, MOSI0, SCK0, MISO1, MOSI1, SCK1, CS0
CAN	CAN0TX, CAN0RX
I2C	SCL1, SCL2, SDA1, SD2

2. Các thiết bị đầu vào

2.1. Module RealTime Clock DS3231

2.1.1. Sơ lược

Hình 2: Module RTC DS3231

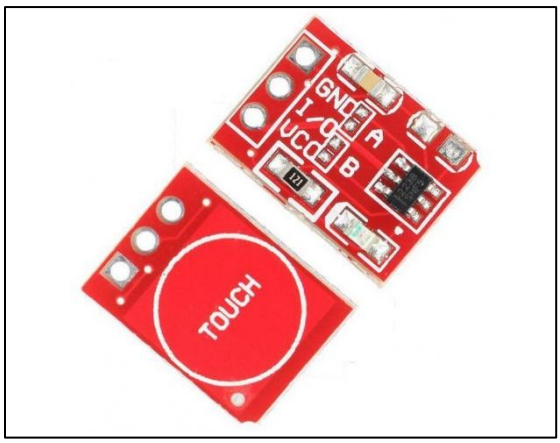
Module Thời Gian Thực RTC DS3231 là IC thời gian thực giá rẻ, rất chính xác với thạch anh tích hợp sẵn có khả năng đo được nhiệt độ. IC có đầu vào cho pin riêng, tách biệt khỏi nguồn chính đảm bảo cho việc giữ thời gian chính xác. Thạch anh tích hợp sẵn giúp tăng độ chính xác trong thời gian dài hoạt động và giảm số lượng linh kiện cần thiết khi làm board. Thời gian trong IC được giữ ở dạng: giờ, phút, giây, ngày, thứ, tháng, năm. Các tháng có ít hơn 31 ngày sẽ tự động được điều chỉnh, các năm Nhuận cũng được chỉnh đúng số ngày. Thời gian có thể hoạt động ở chế độ 24h hoặc 12h AmPM. IC còn có chức năng báo động, có thể cài đặt 2 thời gian báo và lịch, có tín hiệu ra là xung vuông. Giao tiếp với IC được thực hiện thông qua I2C bus.Trong chip có mạch điện áp chuẩn dùng để theo dõi trạng thái của nguồn VCC, phát hiện lỗi nguồn, tự động chuyển nguồn khi có vấn đề. Có tín hiệu Reset xuất ra cho mạch ngoài, MCU khi nguồn điện phục hồi trạng thái.

2.1.2. Thông số kĩ thuật

IC: DS3231
Điện áp hoạt động: 5V
Phương thức truyền: I2C

2.2. Module Touch Sensor TTP223

2.2.1. Sơ lược



Hình 3: *Module Touch Sensor TTP223*

Module Touch Sensor TTP223 thường được sử dụng trong các ứng dụng cảm ứng điện dung: bàn phím, công tắc cảm ứng, báo động,…

Nếu có thao tác chạm cảm ứng chân OUT sẽ xuất tín hiệu và đèn Led sẽ sáng báo hiệu.

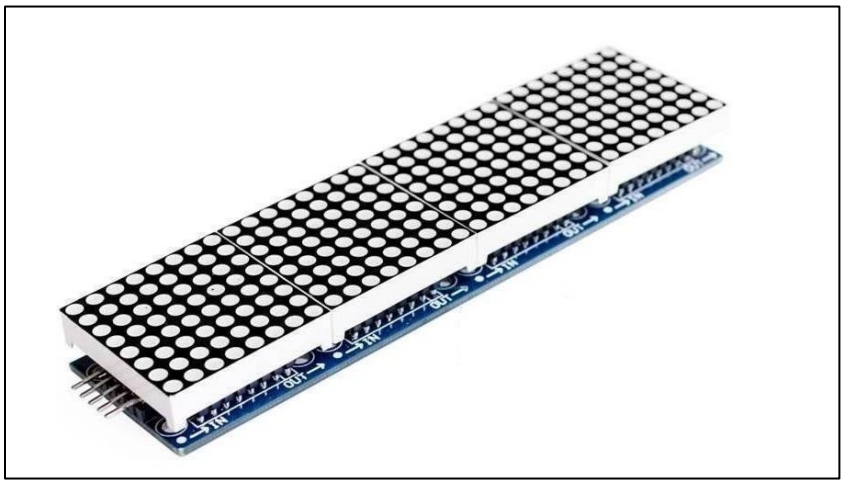
2.2.2.2 Thông số kĩ thuật

IC: TTP223
Điện áp hoạt động: 3 – 5VDC
Phương thức truyền: GPIO

3. Các thiết bị đầu ra

3.1. LED Matrix7219_8x32

3.1.1. Sơ lược



Hình 4: Màn hình Matrix7219_8x32

Module là tổ hợp **4 led Matrix 8x8**, với mỗi led Matrix được điều khiển bởi một IC MAX7219 (tương tự như IC 74HC595). 4 IC MAX7219 được mắc nối tiếp và giao tiếp với nhau bằng chuẩn SPI giúp tiết kiệm số chân cho vi điều khiển (chỉ sử dụng 3 chân để điều khiển). Ngoài ra IC MAX7219 còn được tích hợp static RAM 8x8 từ đó việc hiển thị trở nên mượt mà hơn, đồng thời Module còn có thể mở rộng thêm bằng việc mắc nối tiếp các mạch led MAX7219 lại với nhau.

3.1.2. Thông số kĩ thuật

Điện áp hoạt động:	3.3V/5VDC
Phương thức truyền:	SPI

3.2. Buzzer 5VDC

3.2.1. Sơ lược

Buzzer 5VDC có tuổi thọ cao, hiệu suất ổn định, chất lượng tốt, được sản xuất nhỏ gọn phù hợp thiết kế với các mạch còi buzzer nhỏ gọn, mạch báo động.

Hình 5: Buzzer 5VDC

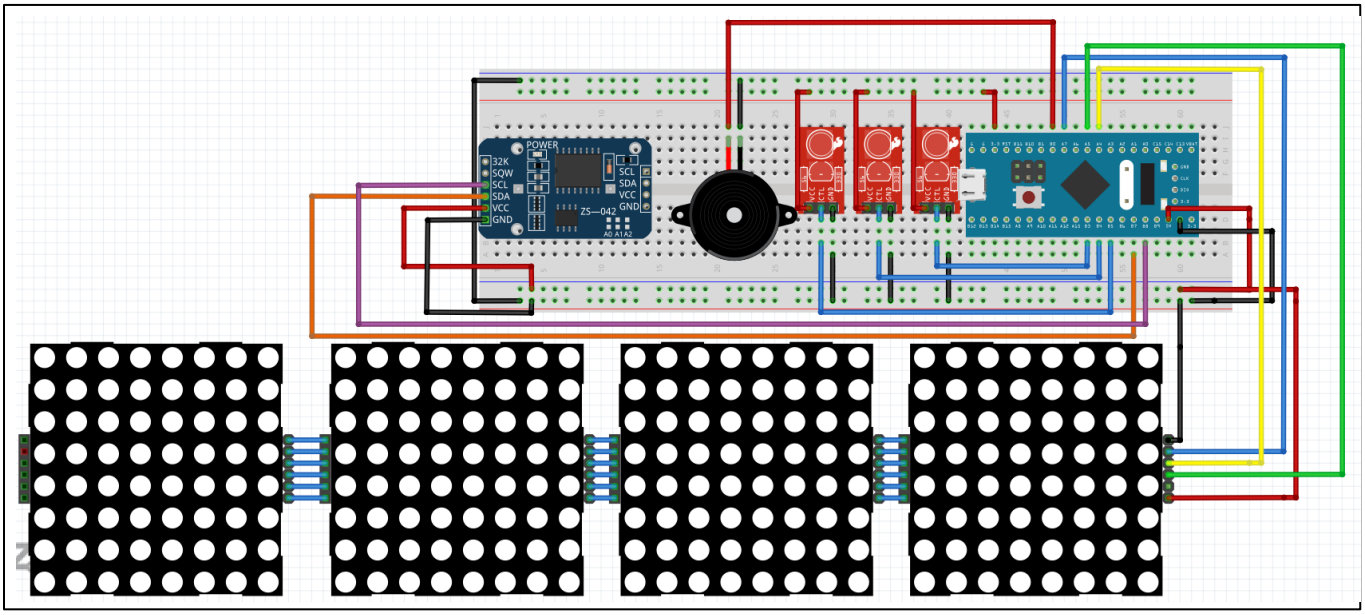
3.2.1. Thông số kĩ thuật

Điện áp hoạt động: 5VDC
Phương thức truyền: GPIO

CHƯƠNG II: THIẾT KẾ, CẤU TẠO VÀ NGUYÊN LÍ HOẠT ĐỘNG

1. *Thiết kế*

1.1. Sơ đồ thiết kế



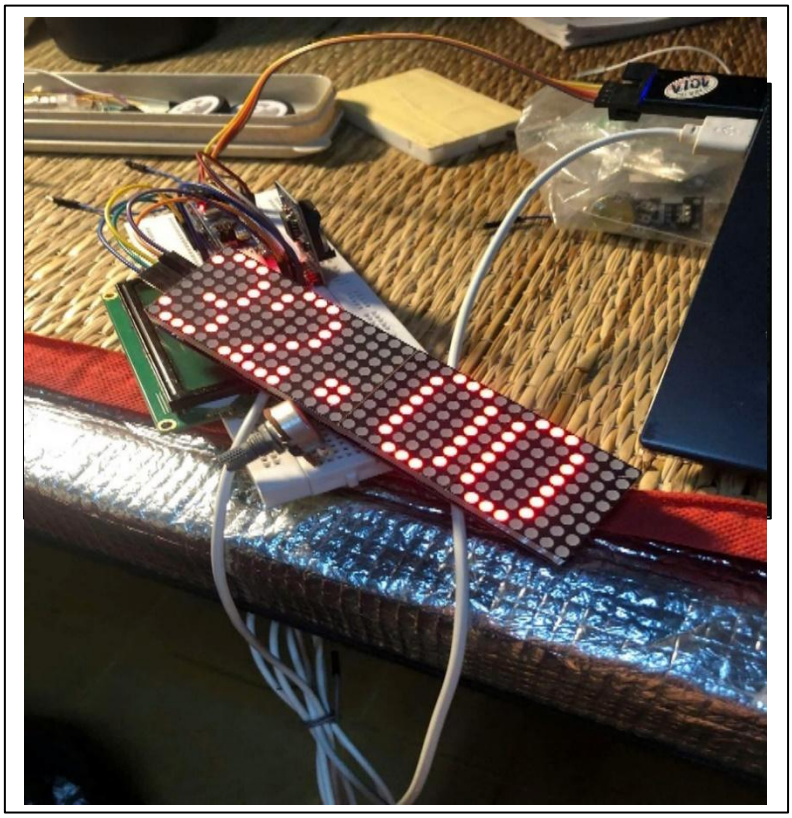
Hình 6: Thiết kế mạch trên Fritzing

Tên linh kiện	Tên chân	Nối vào STM32
Touch 0	VCC	3.3V
	GND	GND
	Out	PB3
Touch 1	VCC	3.3V
	GND	GND
	Out	PB4
Touch 2	VCC	3.3V
	GND	GND
	Out	PB5
DS3231(RTC)	VCC	5V
	GND	GND
	SDA	PB7
	SCL	PB6

Matrix7219_8x32	VCC	5V
	GND	GND
	DIN	PA7
	CLK	PA5
	CS	PA4
Buzzer	VCC	PB0
	GND	GND

Bảng 2: Danh sách nối chân linh kiện

1.2. Hiện thực sản phẩm trên breadboard



Hình 7: Lắp ráp mạch trên breadboard và chạy thử

1.3. Hoàn thiện sản phẩm

Hình 8: Hàn chân và đóng hộp sản phẩm

2. Chương trình nạp

2.1. Thiết lập thư viện

```
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "stdio.h"
#include "Fonts.h"
/* USER CODE END Includes */
```

2.2. *Thư viện dùng cho để tạo kí tự LED Matrix7219_8x32*

```
/*
 * Fonts.h
 *
 *  Created on: Aug 16, 2024
 *      Author: arunrawat
 */

#ifndef INC_FONTS_H_
#define INC_FONTS_H_

#define FONT_WIDTH 8

const uint8_t MAX7219_Dot_Matrix_font [256] [8] = {
  { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 }, // 0x00
  { 0x7E, 0x81, 0x95, 0xB1, 0xB1, 0x95, 0x81, 0x7E }, // 0x01
  { 0x7E, 0xFF, 0xEB, 0xCF, 0xCF, 0xEB, 0xFF, 0x7E }, // 0x02
  { 0x0E, 0x1F, 0x3F, 0x7E, 0x3F, 0x1F, 0x0E, 0x00 }, // 0x03
  { 0x08, 0x1C, 0x3E, 0x7F, 0x3E, 0x1C, 0x08, 0x00 }, // 0x04
  { 0x18, 0xBA, 0xFF, 0xFF, 0xFF, 0xBA, 0x18, 0x00 }, // 0x05
  { 0x10, 0xB8, 0xFC, 0xFF, 0xFC, 0xB8, 0x10, 0x00 }, // 0x06
  { 0x00, 0x00, 0x18, 0x3C, 0x3C, 0x18, 0x00, 0x00 }, // 0x07
  { 0xFF, 0xFF, 0xE7, 0xC3, 0xC3, 0xE7, 0xFF, 0xFF }, // 0x08
  { 0x00, 0x3C, 0x66, 0x42, 0x42, 0x66, 0x3C, 0x00 }, // 0x09
  { 0xFF, 0xC3, 0x99, 0xBD, 0xBD, 0x99, 0xC3, 0xFF }, // 0x0A
  { 0x70, 0xF8, 0x88, 0x88, 0xFD, 0x7F, 0x07, 0x0F }, // 0x0B
  { 0x00, 0x4E, 0x5F, 0xF1, 0xF1, 0x5F, 0x4E, 0x00 }, // 0x0C
  { 0xC0, 0xE0, 0xFF, 0x7F, 0x05, 0x05, 0x07, 0x07 }, // 0x0D
  { 0xC0, 0xFF, 0x7F, 0x05, 0x05, 0x65, 0x7F, 0x3F }, // 0x0E
  { 0x99, 0x5A, 0x3C, 0xE7, 0xE7, 0x3C, 0x5A, 0x99 }, // 0x0F
  { 0x7F, 0x3E, 0x3E, 0x1C, 0x1C, 0x08, 0x08, 0x00 }, // 0x10
  { 0x08, 0x08, 0x1C, 0x1C, 0x3E, 0x3E, 0x7F, 0x00 }, // 0x11
  { 0x00, 0x24, 0x66, 0xFF, 0xFF, 0x66, 0x24, 0x00 }, // 0x12
  { 0x00, 0x5F, 0x5F, 0x00, 0x00, 0x5F, 0x5F, 0x00 }, // 0x13
  { 0x06, 0x0F, 0x09, 0x7F, 0x7F, 0x01, 0x7F, 0x7F }, // 0x14
  { 0x40, 0xDA, 0xBF, 0xA5, 0xFD, 0x59, 0x03, 0x02 }, // 0x15
  { 0x00, 0x70, 0x70, 0x70, 0x70, 0x70, 0x70, 0x00 }, // 0x16
  { 0x80, 0x94, 0xB6, 0xFF, 0xFF, 0xB6, 0x94, 0x80 }, // 0x17
  { 0x00, 0x04, 0x06, 0x7F, 0x7F, 0x06, 0x04, 0x00 }, // 0x18
  { 0x00, 0x10, 0x30, 0x7F, 0x7F, 0x30, 0x10, 0x00 }, // 0x19
  { 0x08, 0x08, 0x08, 0x2A, 0x3E, 0x1C, 0x08, 0x00 }, // 0x1A
  { 0x08, 0x1C, 0x3E, 0x2A, 0x08, 0x08, 0x08, 0x00 }, // 0x1B
  { 0x3C, 0x3C, 0x20, 0x20, 0x20, 0x20, 0x20, 0x00 }, // 0x1C
  { 0x08, 0x1C, 0x3E, 0x08, 0x08, 0x3E, 0x1C, 0x08 }, // 0x1D
  { 0x30, 0x38, 0x3C, 0x3E, 0x3E, 0x3C, 0x38, 0x30 }, // 0x1E
```

```
{ 0x30, 0x38, 0x3C, 0x3E, 0x3E, 0x3C, 0x38, 0x30 }, // 0x1E
  { 0x06, 0x0E, 0x1E, 0x3E, 0x3E, 0x1E, 0x0E, 0x06 }, // 0x1F
  { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 }, // ' '
  { 0x00, 0x06, 0x5F, 0x5F, 0x06, 0x00, 0x00, 0x00 }, // '!'
  { 0x00, 0x07, 0x07, 0x00, 0x07, 0x07, 0x00, 0x00 }, // '"'
  { 0x14, 0x7F, 0x7F, 0x14, 0x7F, 0x7F, 0x14, 0x00 }, // '#'
  { 0x24, 0x2E, 0x6B, 0x6B, 0x3A, 0x12, 0x00, 0x00 }, // '$'
  { 0x46, 0x66, 0x30, 0x18, 0x0C, 0x66, 0x62, 0x00 }, // '%'
  { 0x30, 0x7A, 0x4F, 0x5D, 0x37, 0x7A, 0x48, 0x00 }, // '&'
  { 0x04, 0x07, 0x03, 0x00, 0x00, 0x00, 0x00, 0x00 }, // '''
  { 0x00, 0x1C, 0x3E, 0x63, 0x41, 0x00, 0x00, 0x00 }, // '('
  { 0x00, 0x41, 0x63, 0x3E, 0x1C, 0x00, 0x00, 0x00 }, // ')'
  { 0x08, 0x2A, 0x3E, 0x1C, 0x1C, 0x3E, 0x2A, 0x08 }, // '*'
  { 0x08, 0x08, 0x3E, 0x3E, 0x08, 0x08, 0x00, 0x00 }, // '+'
  { 0x00, 0x80, 0xE0, 0x60, 0x00, 0x00, 0x00, 0x00 }, // ','
  { 0x80, 0x80, 0x80, 0x80, 0x80, 0x00, 0x00, 0x00 }, // '-'
  { 0x00, 0x60, 0x60, 0x00, 0x00, 0x00, 0x00, 0x00 }, // '.'
  { 0x60, 0x30, 0x18, 0x0C, 0x06, 0x03, 0x01, 0x00 }, // '/'
  { 0x7e, 0x81, 0x81, 0x81, 0x7e, 0x00, 0x00, 0x00 }, // '0'
  { 0x84, 0x82, 0xff, 0x80, 0x80, 0x00, 0x00, 0x00 }, // '1'
  { 0x86, 0xc1, 0xa1, 0x91, 0xce, 0x00, 0x00, 0x00 }, // '2'
  { 0x42, 0x81, 0x91, 0x91, 0x6e, 0x00, 0x00, 0x00 }, // '3'
  { 0x30, 0x28, 0x24, 0xff, 0x20, 0x00, 0x00, 0x00 }, // '4'
  { 0x47, 0x85, 0x85, 0x85, 0x79, 0x00, 0x00, 0x00 }, // '5'
  { 0x7e, 0x93, 0x91, 0x91, 0x62, 0x00, 0x00, 0x00 }, // '6'
  { 0x01, 0x01, 0xf1, 0x09, 0x07, 0x00, 0x00, 0x00 }, // '7'
  { 0x76, 0x89, 0x89, 0x89, 0x76, 0x00, 0x00, 0x00 }, // '8'
  { 0x4e, 0x91, 0x91, 0x91, 0x7e, 0x00, 0x00, 0x00 }, // '9'
  { 0x6c, 0x6c, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 }, // ':'
  { 0x00, 0x80, 0xE6, 0x66, 0x00, 0x00, 0x00, 0x00 }, // ';'
  { 0x08, 0x1C, 0x36, 0x63, 0x41, 0x00, 0x00, 0x00 }, // '<'
  { 0x24, 0x24, 0x24, 0x24, 0x24, 0x24, 0x00, 0x00 }, // '='
  { 0x00, 0x41, 0x63, 0x36, 0x1C, 0x08, 0x00, 0x00 }, // '>'
  { 0x02, 0x03, 0x51, 0x59, 0x0F, 0x06, 0x00, 0x00 }, // '?'
  { 0x3E, 0x7F, 0x41, 0x5D, 0x5D, 0x1F, 0x1E, 0x00 }, // '@'
  { 0xfc, 0x26, 0x27, 0x26, 0xfc, 0x00, 0x00, 0x00 }, // 'A'
  { 0x41, 0x7F, 0x7F, 0x49, 0x49, 0x7F, 0x36, 0x00 }, // 'B'
  { 0x1C, 0x3E, 0x63, 0x41, 0x41, 0x63, 0x22, 0x00 }, // 'C'
  { 0x41, 0x7F, 0x7F, 0x41, 0x63, 0x3E, 0x1C, 0x00 }, // 'D'
  { 0x41, 0x7F, 0x7F, 0x49, 0x5D, 0x41, 0x63, 0x00 }, // 'E'
  { 0x41, 0x7F, 0x7F, 0x49, 0x1D, 0x01, 0x03, 0x00 }, // 'F'
  { 0x1C, 0x3E, 0x63, 0x41, 0x51, 0x73, 0x72, 0x00 }, // 'G'
  { 0x7F, 0x7F, 0x08, 0x08, 0x7F, 0x7F, 0x00, 0x00 }, // 'H'
  { 0x00, 0x41, 0x7F, 0x7F, 0x41, 0x00, 0x00, 0x00 }, // 'I'
  { 0x30, 0x70, 0x40, 0x41, 0x7F, 0x3F, 0x01, 0x00 }, // 'J'
  { 0x41, 0x7F, 0x7F, 0x08, 0x1C, 0x77, 0x63, 0x00 }, // 'K'
  { 0xff, 0x80, 0x80, 0x80, 0xc0, 0x00, 0x00, 0x00 }, // 'L'
  { 0x00, 0x7F, 0x0E, 0x1C, 0x0E, 0x7F, 0x00, 0x00 }, // 'M'
  { 0xff, 0x18, 0x30, 0x60, 0xff, 0x00, 0x00, 0x00 }, // 'N'
  { 0x7e, 0x81, 0x81, 0x81, 0x7e, 0x00, 0x00, 0x00 }, // 'O'
  { 0x41, 0x7F, 0x7F, 0x49, 0x09, 0x0F, 0x06, 0x00 }, // 'P'
  { 0x1E, 0x3F, 0x21, 0x71, 0x7F, 0x5E, 0x00, 0x00 }, // 'Q'
  { 0x41, 0x7F, 0x7F, 0x09, 0x19, 0x7F, 0x66, 0x00 }, // 'R'
  { 0x26, 0x6F, 0x4D, 0x59, 0x73, 0x32, 0x00, 0x00 }, // 'S'
```

```
{ 0x7F, 0x7F, 0x40, 0x40, 0x7F, 0x7F, 0x00, 0x00 }, // 'U'
  { 0x1F, 0x3F, 0x60, 0x60, 0x3F, 0x1F, 0x00, 0x00 }, // 'V'
  { 0x7F, 0x7F, 0x30, 0x18, 0x30, 0x7F, 0x7F, 0x00 }, // 'W'
  { 0x43, 0x67, 0x3C, 0x18, 0x3C, 0x67, 0x43, 0x00 }, // 'X'
  { 0x07, 0x4F, 0x78, 0x78, 0x4F, 0x07, 0x00, 0x00 }, // 'Y'
  { 0x47, 0x63, 0x71, 0x59, 0x4D, 0x67, 0x73, 0x00 }, // 'Z'
  { 0x00, 0x7F, 0x7F, 0x41, 0x41, 0x00, 0x00, 0x00 }, // '['
  { 0x01, 0x03, 0x06, 0x0C, 0x18, 0x30, 0x60, 0x00 }, // backslash
  { 0x00, 0x41, 0x41, 0x7F, 0x7F, 0x00, 0x00, 0x00 }, // ']'
  { 0x08, 0x0C, 0x06, 0x03, 0x06, 0x0C, 0x08, 0x00 }, // '^'
  { 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80, 0x80 }, // '_'
  { 0x00, 0x00, 0x03, 0x07, 0x04, 0x00, 0x00, 0x00 }, // '`'
  { 0x20, 0x74, 0x54, 0x54, 0x3C, 0x78, 0x40, 0x00 }, // 'a'
  { 0x41, 0x7F, 0x3F, 0x48, 0x48, 0x78, 0x30, 0x00 }, // 'b'
  { 0x38, 0x7C, 0x44, 0x44, 0x6C, 0x28, 0x00, 0x00 }, // 'c'
  { 0x30, 0x78, 0x48, 0x49, 0x3F, 0x7F, 0x40, 0x00 }, // 'd'
  { 0x38, 0x7C, 0x54, 0x54, 0x5C, 0x18, 0x00, 0x00 }, // 'e'
  { 0x48, 0x7E, 0x7F, 0x49, 0x03, 0x02, 0x00, 0x00 }, // 'f'
  { 0x98, 0xBC, 0xA4, 0xA4, 0xF8, 0x7C, 0x04, 0x00 }, // 'g'
  { 0x41, 0x7F, 0x7F, 0x08, 0x04, 0x7C, 0x78, 0x00 }, // 'h'
  { 0x00, 0x44, 0x7D, 0x7D, 0x40, 0x00, 0x00, 0x00 }, // 'i'
  { 0x60, 0xE0, 0x80, 0x80, 0xFD, 0x7D, 0x00, 0x00 }, // 'j'
  { 0x41, 0x7F, 0x7F, 0x10, 0x38, 0x6C, 0x44, 0x00 }, // 'k'
  { 0x00, 0x41, 0x7F, 0x7F, 0x40, 0x00, 0x00, 0x00 }, // 'l'
  { 0x7C, 0x7C, 0x18, 0x38, 0x1C, 0x7C, 0x78, 0x00 }, // 'm'
  { 0x7C, 0x7C, 0x04, 0x04, 0x7C, 0x78, 0x00, 0x00 }, // 'n'
  { 0x38, 0x7C, 0x44, 0x44, 0x7C, 0x38, 0x00, 0x00 }, // 'o'
  { 0x84, 0xFC, 0xF8, 0xA4, 0x24, 0x3C, 0x18, 0x00 }, // 'p'
  { 0x18, 0x3C, 0x24, 0xA4, 0xF8, 0xFC, 0x84, 0x00 }, // 'q'
  { 0x44, 0x7C, 0x78, 0x4C, 0x04, 0x1C, 0x18, 0x00 }, // 'r'
  { 0x48, 0x5C, 0x54, 0x54, 0x74, 0x24, 0x00, 0x00 }, // 's'
  { 0x00, 0x04, 0x3E, 0x7F, 0x44, 0x24, 0x00, 0x00 }, // 't'
  { 0x3C, 0x7C, 0x40, 0x40, 0x3C, 0x7C, 0x40, 0x00 }, // 'u'
  { 0x1C, 0x3C, 0x60, 0x60, 0x3C, 0x1C, 0x00, 0x00 }, // 'v'
  { 0x3C, 0x7C, 0x70, 0x38, 0x70, 0x7C, 0x3C, 0x00 }, // 'w'
  { 0x44, 0x6C, 0x38, 0x10, 0x38, 0x6C, 0x44, 0x00 }, // 'x'
  { 0x9C, 0xBC, 0xA0, 0xA0, 0xFC, 0x7C, 0x00, 0x00 }, // 'y'
  { 0x4C, 0x64, 0x74, 0x5C, 0x4C, 0x64, 0x00, 0x00 }, // 'z'
  { 0x08, 0x08, 0x3E, 0x77, 0x41, 0x41, 0x00, 0x00 }, // '{'
  { 0x00, 0x00, 0x00, 0x77, 0x77, 0x00, 0x00, 0x00 }, // '|'
  { 0x41, 0x41, 0x77, 0x3E, 0x08, 0x08, 0x00, 0x00 }, // '}'
  { 0x02, 0x03, 0x01, 0x03, 0x02, 0x03, 0x01, 0x00 }, // '~'
  { 0x70, 0x78, 0x4C, 0x46, 0x4C, 0x78, 0x70, 0x00 }, // 0x7F
  { 0x0E, 0x9F, 0x91, 0xB1, 0xFB, 0x4A, 0x00, 0x00 }, // 0x80
  { 0x3A, 0x7A, 0x40, 0x40, 0x7A, 0x7A, 0x40, 0x00 }, // 0x81
  { 0x38, 0x7C, 0x54, 0x55, 0x5D, 0x19, 0x00, 0x00 }, // 0x82
  { 0x02, 0x23, 0x75, 0x55, 0x55, 0x7D, 0x7B, 0x42 }, // 0x83
  { 0x21, 0x75, 0x54, 0x54, 0x7D, 0x79, 0x40, 0x00 }, // 0x84
  { 0x21, 0x75, 0x55, 0x54, 0x7C, 0x78, 0x40, 0x00 }, // 0x85
  { 0x20, 0x74, 0x57, 0x57, 0x7C, 0x78, 0x40, 0x00 }, // 0x86
  { 0x18, 0x3C, 0xA4, 0xA4, 0xE4, 0x40, 0x00, 0x00 }, // 0x87
  { 0x02, 0x3B, 0x7D, 0x55, 0x55, 0x5D, 0x1B, 0x02 }, // 0x88
  { 0x39, 0x7D, 0x54, 0x54, 0x5D, 0x19, 0x00, 0x00 }, // 0x89
  { 0x39, 0x7D, 0x55, 0x54, 0x5C, 0x18, 0x00, 0x00 }, // 0x8A
  { 0x01, 0x45, 0x7C, 0x7C, 0x41, 0x01, 0x00, 0x00 }, // 0x8B
```

```
{ 0x02, 0x03, 0x45, 0x7D, 0x7D, 0x43, 0x02, 0x00 }, // 0x8C
{ 0x01, 0x45, 0x7D, 0x7C, 0x40, 0x00, 0x00, 0x00 }, // 0x8D
{ 0x79, 0x7D, 0x16, 0x12, 0x16, 0x7D, 0x79, 0x00 }, // 0x8E
{ 0x70, 0x78, 0x2B, 0x2B, 0x78, 0x70, 0x00, 0x00 }, // 0x8F
{ 0x44, 0x7C, 0x7C, 0x55, 0x55, 0x45, 0x00, 0x00 }, // 0x90
{ 0x20, 0x74, 0x54, 0x54, 0x7C, 0x7C, 0x54, 0x54 }, // 0x91
{ 0x7C, 0x7E, 0x0B, 0x09, 0x7F, 0x7F, 0x49, 0x00 }, // 0x92
{ 0x32, 0x7B, 0x49, 0x49, 0x7B, 0x32, 0x00, 0x00 }, // 0x93
{ 0x32, 0x7A, 0x48, 0x48, 0x7A, 0x32, 0x00, 0x00 }, // 0x94
{ 0x32, 0x7A, 0x4A, 0x48, 0x78, 0x30, 0x00, 0x00 }, // 0x95
{ 0x3A, 0x7B, 0x41, 0x41, 0x7B, 0x7A, 0x40, 0x00 }, // 0x96
{ 0x3A, 0x7A, 0x42, 0x40, 0x78, 0x78, 0x40, 0x00 }, // 0x97
{ 0x9A, 0xBA, 0xA0, 0xA0, 0xFA, 0x7A, 0x00, 0x00 }, // 0x98
{ 0x01, 0x19, 0x3C, 0x66, 0x66, 0x3C, 0x19, 0x01 }, // 0x99
{ 0x3D, 0x7D, 0x40, 0x40, 0x7D, 0x3D, 0x00, 0x00 }, // 0x9A
{ 0x18, 0x3C, 0x24, 0xE7, 0xE7, 0x24, 0x24, 0x00 }, // 0x9B
{ 0x68, 0x7E, 0x7F, 0x49, 0x43, 0x66, 0x20, 0x00 }, // 0x9C
{ 0x2B, 0x2F, 0xFC, 0xFC, 0x2F, 0x2B, 0x00, 0x00 }, // 0x9D
{ 0xFF, 0xFF, 0x09, 0x09, 0x2F, 0xF6, 0xF8, 0xA0 }, // 0x9E
{ 0x40, 0xC0, 0x88, 0xFE, 0x7F, 0x09, 0x03, 0x02 }, // 0x9F
{ 0x20, 0x74, 0x54, 0x55, 0x7D, 0x79, 0x40, 0x00 }, // 0xA0
{ 0x00, 0x44, 0x7D, 0x7D, 0x41, 0x00, 0x00, 0x00 }, // 0xA1
{ 0x30, 0x78, 0x48, 0x4A, 0x7A, 0x32, 0x00, 0x00 }, // 0xA2
{ 0x38, 0x78, 0x40, 0x42, 0x7A, 0x7A, 0x40, 0x00 }, // 0xA3
{ 0x7A, 0x7A, 0x0A, 0x0A, 0x7A, 0x70, 0x00, 0x00 }, // 0xA4
{ 0x7D, 0x7D, 0x19, 0x31, 0x7D, 0x7D, 0x00, 0x00 }, // 0xA5
{ 0x00, 0x26, 0x2F, 0x29, 0x2F, 0x2F, 0x28, 0x00 }, // 0xA6
{ 0x00, 0x26, 0x2F, 0x29, 0x2F, 0x26, 0x00, 0x00 }, // 0xA7
{ 0x30, 0x78, 0x4D, 0x45, 0x60, 0x20, 0x00, 0x00 }, // 0xA8
{ 0x38, 0x38, 0x08, 0x08, 0x08, 0x08, 0x00, 0x00 }, // 0xA9
{ 0x08, 0x08, 0x08, 0x08, 0x38, 0x38, 0x00, 0x00 }, // 0xAA
{ 0x4F, 0x6F, 0x30, 0x18, 0xCC, 0xEE, 0xBB, 0x91 }, // 0xAB
{ 0x4F, 0x6F, 0x30, 0x18, 0x6C, 0x76, 0xFB, 0xF9 }, // 0xAC
{ 0x00, 0x00, 0x00, 0x7B, 0x7B, 0x00, 0x00, 0x00 }, // 0xAD
{ 0x08, 0x1C, 0x36, 0x22, 0x08, 0x1C, 0x36, 0x22 }, // 0xAE
{ 0x22, 0x36, 0x1C, 0x08, 0x22, 0x36, 0x1C, 0x08 }, // 0xAF
{ 0xAA, 0x00, 0x55, 0x00, 0xAA, 0x00, 0x55, 0x00 }, // 0xB0
{ 0xAA, 0x55, 0xAA, 0x55, 0xAA, 0x55, 0xAA, 0x55 }, // 0xB1
{ 0xDD, 0xFF, 0xAA, 0x77, 0xDD, 0xAA, 0xFF, 0x77 }, // 0xB2
{ 0x00, 0x00, 0x00, 0xFF, 0xFF, 0x00, 0x00, 0x00 }, // 0xB3
{ 0x10, 0x10, 0x10, 0xFF, 0xFF, 0x00, 0x00, 0x00 }, // 0xB4
{ 0x14, 0x14, 0x14, 0xFF, 0xFF, 0x00, 0x00, 0x00 }, // 0xB5
{ 0x10, 0x10, 0xFF, 0xFF, 0x00, 0xFF, 0xFF, 0x00 }, // 0xB6
{ 0x10, 0x10, 0xF0, 0xF0, 0x10, 0xF0, 0xF0, 0x00 }, // 0xB7
{ 0x14, 0x14, 0x14, 0xFC, 0xFC, 0x00, 0x00, 0x00 }, // 0xB8
{ 0x14, 0x14, 0xF7, 0xF7, 0x00, 0xFF, 0xFF, 0x00 }, // 0xB9
{ 0x00, 0x00, 0xFF, 0xFF, 0x00, 0xFF, 0xFF, 0x00 }, // 0xBA
{ 0x14, 0x14, 0xF4, 0xF4, 0x04, 0xFC, 0xFC, 0x00 }, // 0xBB
{ 0x14, 0x14, 0x17, 0x17, 0x10, 0x1F, 0x1F, 0x00 }, // 0xBC
{ 0x10, 0x10, 0x1F, 0x1F, 0x10, 0x1F, 0x1F, 0x00 }, // 0xBD
{ 0x14, 0x14, 0x14, 0x1F, 0x1F, 0x00, 0x00, 0x00 }, // 0xBE
{ 0x10, 0x10, 0x10, 0xF0, 0xF0, 0x00, 0x00, 0x00 }, // 0xBF
{ 0x00, 0x00, 0x00, 0x1F, 0x1F, 0x10, 0x10, 0x10 }, // 0xC0
{ 0x10, 0x10, 0x10, 0x1F, 0x1F, 0x10, 0x10, 0x10 }, // 0xC1
{ 0x10, 0x10, 0x10, 0xF0, 0xF0, 0x10, 0x10, 0x10 }, // 0xC2
```

```
{ 0x00, 0x00, 0x00, 0xFF, 0xFF, 0x10, 0x10, 0x10 }, // 0xC3
  { 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10, 0x10 }, // 0xC4
  { 0x10, 0x10, 0x10, 0xFF, 0xFF, 0x10, 0x10, 0x10 }, // 0xC5
  { 0x00, 0x00, 0x00, 0xFF, 0xFF, 0x14, 0x14, 0x14 }, // 0xC6
  { 0x00, 0x00, 0xFF, 0xFF, 0x00, 0xFF, 0xFF, 0x10 }, // 0xC7
  { 0x00, 0x00, 0x1F, 0x1F, 0x10, 0x17, 0x17, 0x14 }, // 0xC8
  { 0x00, 0x00, 0xFC, 0xFC, 0x04, 0xF4, 0xF4, 0x14 }, // 0xC9
  { 0x14, 0x14, 0x17, 0x17, 0x10, 0x17, 0x17, 0x14 }, // 0xCA
  { 0x14, 0x14, 0xF4, 0xF4, 0x04, 0xF4, 0xF4, 0x14 }, // 0xCB
  { 0x00, 0x00, 0xFF, 0xFF, 0x00, 0xF7, 0xF7, 0x14 }, // 0xCC
  { 0x14, 0x14, 0x14, 0x14, 0x14, 0x14, 0x14, 0x14 }, // 0xCD
  { 0x14, 0x14, 0xF7, 0xF7, 0x00, 0xF7, 0xF7, 0x14 }, // 0xCE
  { 0x14, 0x14, 0x14, 0x17, 0x17, 0x14, 0x14, 0x14 }, // 0xCF
  { 0x10, 0x10, 0x1F, 0x1F, 0x10, 0x1F, 0x1F, 0x10 }, // 0xD0
  { 0x14, 0x14, 0x14, 0xF4, 0xF4, 0x14, 0x14, 0x14 }, // 0xD1
  { 0x10, 0x10, 0xF0, 0xF0, 0x10, 0xF0, 0xF0, 0x10 }, // 0xD2
  { 0x00, 0x00, 0x1F, 0x1F, 0x10, 0x1F, 0x1F, 0x10 }, // 0xD3
  { 0x00, 0x00, 0x00, 0x1F, 0x1F, 0x14, 0x14, 0x14 }, // 0xD4
  { 0x00, 0x00, 0x00, 0xFC, 0xFC, 0x14, 0x14, 0x14 }, // 0xD5
  { 0x00, 0x00, 0xF0, 0xF0, 0x10, 0xF0, 0xF0, 0x10 }, // 0xD6
  { 0x10, 0x10, 0xFF, 0xFF, 0x10, 0xFF, 0xFF, 0x10 }, // 0xD7
  { 0x14, 0x14, 0x14, 0xFF, 0xFF, 0x14, 0x14, 0x14 }, // 0xD8
  { 0x10, 0x10, 0x10, 0x1F, 0x1F, 0x00, 0x00, 0x00 }, // 0xD9
  { 0x00, 0x00, 0x00, 0xF0, 0xF0, 0x10, 0x10, 0x10 }, // 0xDA
  { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF }, // 0xDB
  { 0xF0, 0xF0, 0xF0, 0xF0, 0xF0, 0xF0, 0xF0, 0xF0 }, // 0xDC
  { 0xFF, 0xFF, 0xFF, 0xFF, 0x00, 0x00, 0x00, 0x00 }, // 0xDD
  { 0x00, 0x00, 0x00, 0x00, 0xFF, 0xFF, 0xFF, 0xFF }, // 0xDE
  { 0x0F, 0x0F, 0x0F, 0x0F, 0x0F, 0x0F, 0x0F, 0x0F }, // 0xDF
  { 0x38, 0x7C, 0x44, 0x6C, 0x38, 0x6C, 0x44, 0x00 }, // 0xE0
  { 0xFC, 0xFE, 0x2A, 0x2A, 0x3E, 0x14, 0x00, 0x00 }, // 0xE1
  { 0x7E, 0x7E, 0x02, 0x02, 0x06, 0x06, 0x00, 0x00 }, // 0xE2
  { 0x02, 0x7E, 0x7E, 0x02, 0x7E, 0x7E, 0x02, 0x00 }, // 0xE3
  { 0x63, 0x77, 0x5D, 0x49, 0x63, 0x63, 0x00, 0x00 }, // 0xE4
  { 0x38, 0x7C, 0x44, 0x7C, 0x3C, 0x04, 0x04, 0x00 }, // 0xE5
  { 0x80, 0xFE, 0x7E, 0x20, 0x20, 0x3E, 0x1E, 0x00 }, // 0xE6
  { 0x04, 0x06, 0x02, 0x7E, 0x7C, 0x06, 0x02, 0x00 }, // 0xE7
  { 0x99, 0xBD, 0xE7, 0xE7, 0xBD, 0x99, 0x00, 0x00 }, // 0xE8
  { 0x1C, 0x3E, 0x6B, 0x49, 0x6B, 0x3E, 0x1C, 0x00 }, // 0xE9
  { 0x4C, 0x7E, 0x73, 0x01, 0x73, 0x7E, 0x4C, 0x00 }, // 0xEA
  { 0x30, 0x78, 0x4A, 0x4F, 0x7D, 0x39, 0x00, 0x00 }, // 0xEB
  { 0x18, 0x3C, 0x24, 0x3C, 0x3C, 0x24, 0x3C, 0x18 }, // 0xEC
  { 0x98, 0xFC, 0x64, 0x3C, 0x3E, 0x27, 0x3D, 0x18 }, // 0xED
  { 0x1C, 0x3E, 0x6B, 0x49, 0x49, 0x00, 0x00, 0x00 }, // 0xEE
  { 0x7E, 0x7F, 0x01, 0x01, 0x7F, 0x7E, 0x00, 0x00 }, // 0xEF
  { 0x2A, 0x2A, 0x2A, 0x2A, 0x2A, 0x2A, 0x00, 0x00 }, // 0xF0
  { 0x44, 0x44, 0x5F, 0x5F, 0x44, 0x44, 0x00, 0x00 }, // 0xF1
  { 0x40, 0x51, 0x5B, 0x4E, 0x44, 0x40, 0x00, 0x00 }, // 0xF2
  { 0x40, 0x44, 0x4E, 0x5B, 0x51, 0x40, 0x00, 0x00 }, // 0xF3
  { 0x00, 0x00, 0x00, 0xFE, 0xFF, 0x01, 0x07, 0x06 }, // 0xF4
  { 0x60, 0xE0, 0x80, 0xFF, 0x7F, 0x00, 0x00, 0x00 }, // 0xF5
  { 0x08, 0x08, 0x6B, 0x6B, 0x08, 0x08, 0x00, 0x00 }, // 0xF6
  { 0x24, 0x36, 0x12, 0x36, 0x24, 0x36, 0x12, 0x00 }, // 0xF7
  { 0x00, 0x06, 0x0F, 0x09, 0x0F, 0x06, 0x00, 0x00 }, // 0xF8
  { 0x00, 0x00, 0x00, 0x18, 0x18, 0x00, 0x00, 0x00 }, // 0xF9
```

```
  { 0x00, 0x00, 0x00, 0x10, 0x10, 0x00, 0x00, 0x00 }, // 0xFA
  { 0x10, 0x30, 0x70, 0xC0, 0xFF, 0xFF, 0x01, 0x01 }, // 0xFB
  { 0x00, 0x1F, 0x1F, 0x01, 0x1F, 0x1E, 0x00, 0x00 }, // 0xFC
  { 0x00, 0x19, 0x1D, 0x17, 0x12, 0x00, 0x00, 0x00 }, // 0xFD
  { 0x00, 0x00, 0x3C, 0x3C, 0x3C, 0x3C, 0x00, 0x00 }, // 0xFE
  { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 }, // 0xFF
};  //  end of MAX7219_Dot_Matrix_font

#endif /* INC_FONTS_H_ */
```

2.3. Tạo Struct và định nghĩa hàm cho DS3231 và LED MaTrix 8x32

```
#define NUM_DEV 4 // sử dụng 4 module max7219
uint8_t bufferCol[NUM_DEV*8]; //kích thước: 32 bytes, dùng để lưu trạng thái hiển thị
của ma trận 8x32
//cách lưu trữ:
//Mỗi byte trong bufferCol lưu trạng thái của một cột gồm 8 LED theo chiều dọc.
//Tổng cộng có 32 cột, tương ứng với 4 module MAX7219, mỗi module có 8 cột.
void max7219_cmd(uint8_t Addr, uint8_t data)
{
     uint16_t writeData = (Addr<<8)|data; //dia chi bat dau tu bit thu 8
     HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, 0); //bat che do slave
     for(int i =0; i< NUM_DEV; i++){
           HAL_SPI_Transmit(&hspi1, (uint8_t *)&writeData, 1, 100);
     }
     HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, 1); //tat che do slave
}

void max_write(int row, uint8_t data){
     int devTarget = (row -1 )/8; //find out which is the actual max, where we need
to write the data
     int offset = devTarget *8; // the offset of the start byte for the max connected
     uint16_t writeData = 0;

     HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, 0); //enable the slave
     for(int dev = 0; dev < NUM_DEV; dev++)//for loop for all the max connected
     {
           if(dev == devTarget)// if this the target
           {
           writeData = ((row - offset)<<8)|data; //send the column number and the
data byte
           HAL_SPI_Transmit(&hspi1, (uint8_t*)&writeData, 1, 1000);
           }else{
                 writeData = 0; //else send NO-OP
                 HAL_SPI_Transmit(&hspi1, (uint8_t *)&writeData, 1, 1000);
           }
     }
     HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, 1); //disable slave mode
}
```

```
//chuyen doi row sang collumn
void flushBuffer (void)
{
     uint8_t bufferRow[NUM_DEV*8] = {0};  // buffer to store data column wise

     /* Convert Cols to Rows */
     for (int i=0; i<NUM_DEV*8; i++)  // for loop for all the bytes
     {
           int dev = i/8;  // 0,1,2,3..  // keep track of which max is being written
           for (int j=0; j<8; j++)  // for loop to extract bits
           {
                 if  ((bufferCol[i])&(1<<(j)))    //  if  the  bit  is  1  //  start
extracting from the 0th bit of C0
                 {
                       bufferRow[j+(8*dev)]  |=  (1<<(7-(i-(8*dev))));    //  start
writing it from the 7th bit of R0
                 }
           }
     }

     /* send Row Data */
     for (int row=1; row<=(NUM_DEV*8); row++)  // write the column data into the
columns
     {
           max_write(row, bufferRow[row-1]);
     }
}
void matrixInit(void)
{
     max7219_cmd(0x09, 0); //no decode, dua tren truthtable cua max7219
     max7219_cmd(0x0a, 0x01); // 3/32 itensity (cuong do sang)
     max7219_cmd(0x0B, 0x07); //scan limit mode, cho phep quet 8 cot tren 1 con LED
matrix
     max7219_cmd(0x0C,  0x01);  //shutdown  register  mode,  1  la  bat  che  do  normal
operation mode
     max7219_cmd(0x0F, 0); //no display test
}
void clearDisplay(void){
     for(int i = 0; i<NUM_DEV*8-1; i++){
           bufferCol[i] = 0;
     }
     flushBuffer();
}
void shiftLeft(void){
     for(int cnt = NUM_DEV*8-2; cnt>=0; cnt--){
           bufferCol[cnt+1] = bufferCol[cnt];
     }
     bufferCol[0]=0;
     flushBuffer();
}
```

```
void shiftchar (uint8_t ch, int delay)
{
     int indx=0;
     for (int i=0; i<FONT_WIDTH-1; i++)  // loop for all the bytes of the font
     {
           uint8_t data = 0;
           /* Chnage the order of the bits */
           for (int j=7; j>=0; j--)  // extract bits from a single byte
           {
                 if ((MAX7219_Dot_Matrix_font[ch][indx])&(1<<j))  // if the bit is
1 // start extracting from the 7th bit of byte
                 {
                       data |= (1<<(7-j));  // start writing it from the 0th bit of
data
                 }
           }
           bufferCol[0] = data;  // store the modified byte to the first element
only. It will shift later
           flushBuffer();
           shiftLeft();
           indx++;
           HAL_Delay(delay);
     }
    flushBuffer();
}
void scrollString (char *str, int delay)
{
     while (*str)
     {
           shiftchar(*str, delay);
           str++;
     }
}
void printString (uint8_t *str)
{
     int strindx = 0;
     for (int k = NUM_DEV*8-1; k>=0; )
     {
           int indx=0;
           for (int i=0; i<FONT_WIDTH-2; i++)  // loop for all the bytes of the
font
           {
                 uint8_t data = 0;
                 /* Chnage the order of the bits */
                 for (int j=7; j>=0; j--)  // extract bits from a single byte
                 {
                       if   ((MAX7219_Dot_Matrix_font[str[strindx]][indx])&(1<<j))
// if the bit is 1 // start extracting from the 7th bit of byte
                       {
                             data |= (1<<(7-j));  // start writing it from the 0th
bit of data
                       }
                 }
                 bufferCol[k--] = data;  // store the modified byte to the first
element only. It will shift later
                 indx++;
```

```
            }
            strindx++;
      }
      flushBuffer();
}

// su dung ds3231
#define DS3231_ADDRESS 0xD0

// Convert normal decimal numbers to binary coded decimal
uint8_t decToBcd(int val)
{
  return (uint8_t)( (val/10*16) + (val%10) );
}

// Convert binary coded decimal to normal decimal numbers
int bcdToDec(uint8_t val)
{
  return (int)( (val/16*10) + (val%16) );
}

typedef struct{
      uint8_t seconds;
      uint8_t minutes;
      uint8_t hour;
      uint8_t dayofmonth;
      uint8_t dayofweek;
      uint8_t month;
      uint8_t year;
} TIME;
TIME time;//khai báo biến time dạng TIME

/* function to set time */

void Set_Time (uint8_t sec, uint8_t min, uint8_t hour, uint8_t dow, uint8_t dom,
uint8_t month, uint8_t year)
{ //dow = day of week, dom = day of month
      uint8_t set_time[7];
      set_time[0] = decToBcd(sec);
      set_time[1] = decToBcd(min);
      set_time[2] = decToBcd(hour);
      set_time[3] = decToBcd(dow);
      set_time[4] = decToBcd(dom);
      set_time[5] = decToBcd(month);
      set_time[6] = decToBcd(year);

      HAL_I2C_Mem_Write(&hi2c1, DS3231_ADDRESS, 0x00, 1, set_time, 7, 1000);
}
```

```
void Get_Time (void)
{
     uint8_t get_time[7];
     HAL_I2C_Mem_Read(&hi2c1, DS3231_ADDRESS, 0x00, 1, get_time, 7, 1000);
     time.seconds = bcdToDec(get_time[0]);
     time.minutes = bcdToDec(get_time[1]);
     time.hour = bcdToDec(get_time[2]);
     time.dayofweek = bcdToDec(get_time[3]);
     time.dayofmonth = bcdToDec(get_time[4]);
     time.month = bcdToDec(get_time[5]);
     time.year = bcdToDec(get_time[6]);
}
float Get_Temp (void)
{
     uint8_t temp[2];
     HAL_I2C_Mem_Read(&hi2c1, DS3231_ADDRESS, 0x11, 1, temp, 2, 1000); //0x11 la
dia chi doc nhiet do
     return ((temp[0])+(temp[1]>>6)/4.0);
}
void force_temp_conv (void)
{
     uint8_t status=0;
     uint8_t control=0;
     HAL_I2C_Mem_Read(&hi2c1, DS3231_ADDRESS, 0x0F, 1, &status, 1, 100);  // read
status register
     if (!(status&0x04))  // if the BSY bit is not set
     {
           HAL_I2C_Mem_Read(&hi2c1, DS3231_ADDRESS, 0x0E, 1, &control, 1, 100);  //
read control register
           HAL_I2C_Mem_Write(&hi2c1,    DS3231_ADDRESS,    0x0E,    1,    (uint8_t
*)(control|(0x20)), 1, 100);  // write modified control register with CONV bit as'1'
     }
}
```

2.4. Tạo ngắt dùng cho 3 nút nhấn PB3, PB4, PB5 (tương ứng 3 MODE 1, 2, 3)

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
     if(GPIO_Pin == GPIO_PIN_3){ // Nút Mode 1
           flag = 1;
           if (alarm_45min_active == 1) {
                 alarm_45min_active = 0;
                 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); // Tắt LED
                 alarm_led_on = 0; // Reset cờ LED
                 last_triggered_total_minutes = 0xFFFF; // Reset chống lặp
           }
     } else if(GPIO_Pin == GPIO_PIN_4){ // Nút Mode 2
           flag = 2;
           if (alarm_45min_active == 1) {
                 alarm_45min_active = 0;
                 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); // Tắt LED
                 alarm_led_on = 0; // Reset cờ LED
                 last_triggered_total_minutes = 0xFFFF; // Reset chống lặp
           }
     } else if(GPIO_Pin == GPIO_PIN_5){ // Nút Mode 3 (Bật/Tắt báo thức)
        if (alarm_45min_active == 0) {
             flag = 3; // Đặt flag để main loop xử lý bật
        } else {
            // Nếu đang bật, nhấn nút này sẽ tắt
            alarm_45min_active = 0;
            HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); // Tắt LED
            alarm_led_on = 0; // Reset cờ LED
            last_triggered_total_minutes = 0xFFFF; // Reset chống lặp
            flag = 1; // Quay về hiển thị giờ
        }
     }
}
```

2.5. Khai báo biến và khởi tạo các init

```
uint16_t count=0;
uint16_tminu=0,min=0,sec=0,hour=0;
HAL_Init();SystemClock
_Config();MX_GPIO_Init
();MX_I2C1_Init();

LCD_init();LCD_clear()
;LCD_setCursor(0,3);
LCD_printf("powered
by");LCD_setCursor(1,
1);LCD_printf("BERNOULLITeam");
HAL_Delay(2000);DS1307_config()
;
 //DS1307_settime(00,54,4,4,27,3,2024);
```

```
/* USER CODE BEGIN PV */
char buffer[20];
float TEMP;
volatile uint8_t flag = 1; // Chế độ hiển thị hiện tại (1: Time, 2: Temp)

// Biến cho chức năng báo thức 45 phút
volatile uint8_t alarm_45min_active = 0; // 0: Tắt, 1: Bật
uint16_t next_alarm_total_minutes = 0; // Tổng số phút (từ 00:00) của lần báo
thức tiếp theo
uint16_t last_triggered_total_minutes = 0xFFFF; // Chống lặp trigger trong
cùng 1 phút

// Biến quản lý LED PB0
volatile uint8_t alarm_led_on = 0;    // Cờ báo LED PB0 đang BẬT
uint32_t led_on_timestamp = 0;       // Mốc thời gian khi bật LED (dùng
HAL_GetTick())

const uint32_t LED_ON_DURATION = 5000; // Thời gian LED sáng (5000ms = 5 giây)
/* USER CODE END PV */
/* USER CODE BEGIN 2 */
  matrixInit();
  clearDisplay();
//  Set_Time(00, 35, 00, 6, 19, 4, 25); //set thời gian theo ý muốn, sau khi
nạp lại sẽ xóa dòng này
  /* USER CODE END 2 */
```

2.6. MODE1: Hiển thị thời gian, MODE2: Hiển thị nhiệt độ, MODE3: Báo Thức (Sau 45p sẽ báo thức)

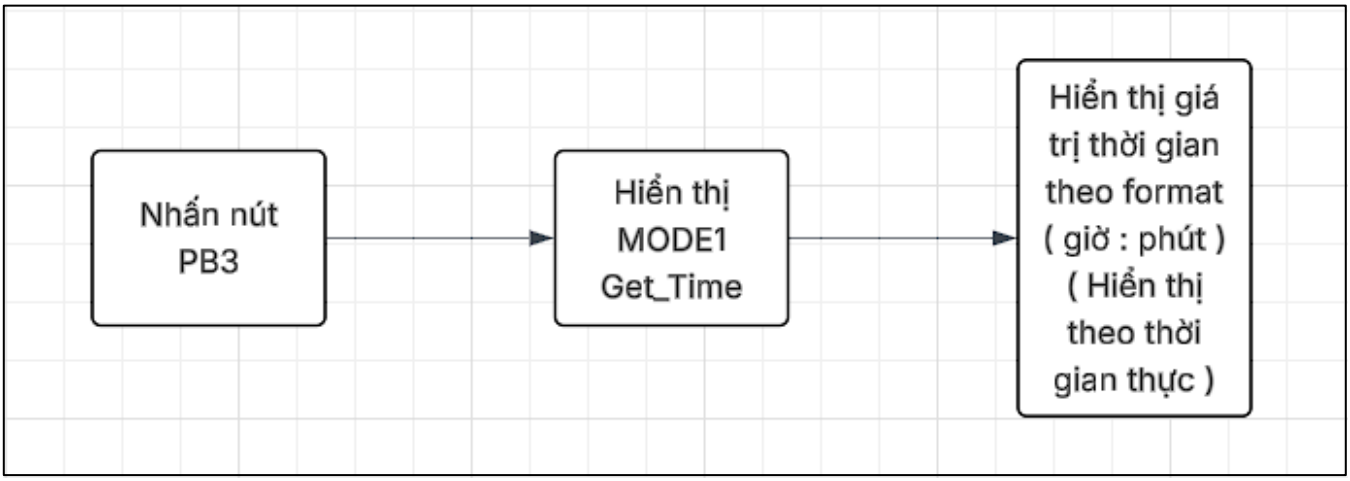
```
/* USER CODE BEGIN 3 */
       uint32_t current_tick = HAL_GetTick();

       // 1. Xử lý kích hoạt báo thức khi nhấn nút Mode 3 (PB5)
       if (flag == 3) {
           Get_Time();
           uint16_t start_total_minutes = (uint16_t)time.hour * 60 +
time.minutes;
           // Tính thời điểm báo thức đầu tiên (sau 45 phút)
           next_alarm_total_minutes = (start_total_minutes + 45) % (24 * 60);
// Dùng 45 phút
           alarm_45min_active = 1;       // Bật cờ báo thức
           alarm_led_on = 0;             // Reset cờ LED ON
           last_triggered_total_minutes = 0xFFFF; // Reset biến chống lặp
           HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); // Đảm bảo
LED tắt ban đầu
           flag = 1;// Quay lại chế độ hiển thị giờ
       }

       // 2. Hiển thị theo chế độ (flag 1 hoặc 2) và kiểm tra trigger
       if (flag == 1) { // Hiển thị thời gian
           Get_Time();
           if (alarm_45min_active == 1) {
               sprintf(buffer,"%02d:%02d", time.minutes,time.seconds);//hiển
thị bộ đếm theo tg thực
               printString((uint8_t*)buffer);
           } else {
               sprintf(buffer,"%02d:%02d", time.hour, time.minutes); // Hiển
thị giờ
               printString((uint8_t*)buffer);
           }

           // Kiểm tra trigger báo thức nếu đang bật
           if (alarm_45min_active == 1) {
               uint16_t current_total_minutes = (uint16_t)time.hour * 60 +
time.minutes;
               if (current_total_minutes == next_alarm_total_minutes &&
current_total_minutes != last_triggered_total_minutes) {
                   if (alarm_led_on == 0) { // Chỉ bật nếu LED đang tắt
                       HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET); //
Bật LED PB0
                       printString("AL_ON");
                       HAL_Delay(5000);
                       alarm_led_on = 1;                     // Đặt cờ LED
đang bật
                       led_on_timestamp = current_tick;      // Ghi lại thời
điểm bật LED
                       last_triggered_total_minutes = current_total_minutes;
// Đánh dấu phút này đã trigger
                       // Tính toán thời điểm báo thức tiếp theo (sau 45 phút
nữa)
```

```
                        next_alarm_total_minutes = (current_total_minutes + 45)
% (24 * 60); // Dùng 45 phút
                    }
                }
            }
        } else if (flag == 2) { // Hiển thị nhiệt độ
            force_temp_conv();
            TEMP = Get_Temp();
            sprintf(buffer, "%.2f", TEMP);
            printString((uint8_t*)buffer);
        }

        // 3. Xử lý tắt LED sau 5 giây (chạy mỗi vòng lặp)
        if (alarm_led_on == 1) {
            // Kiểm tra nếu đã đủ 5000ms (5 giây) kể từ lúc bật
            if (current_tick - led_on_timestamp >= LED_ON_DURATION) {
                HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_RESET); // Tắt
LED PB0
                alarm_led_on = 0; // Reset cờ LED
            }
        }

        HAL_Delay(10); // Giữ delay nhỏ
}
  /* USER CODE END 3 */
```

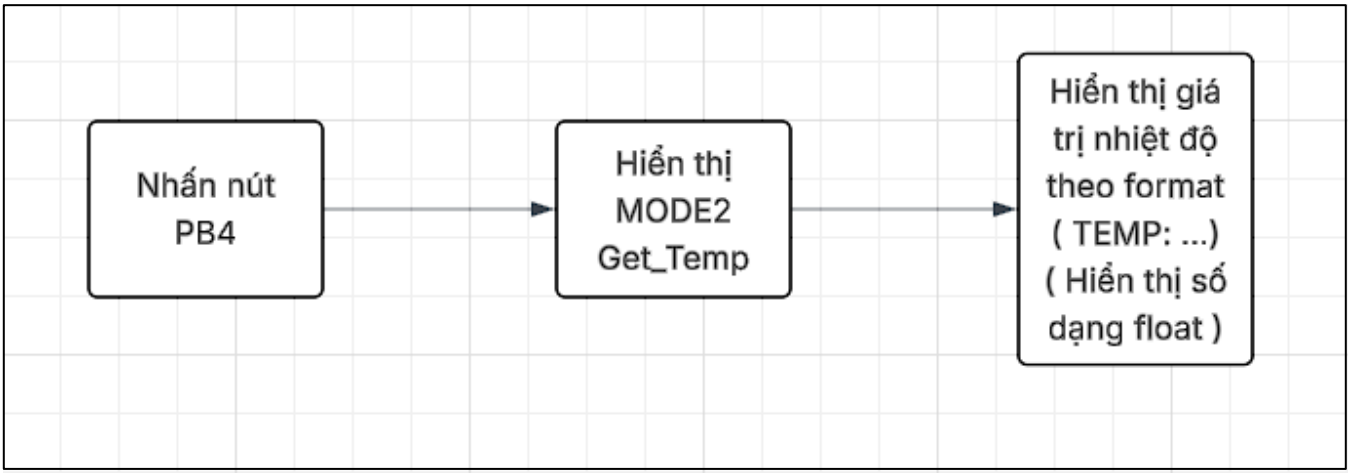
Link demo

3. *Nguyên lí hoạt động theo Block Chart*

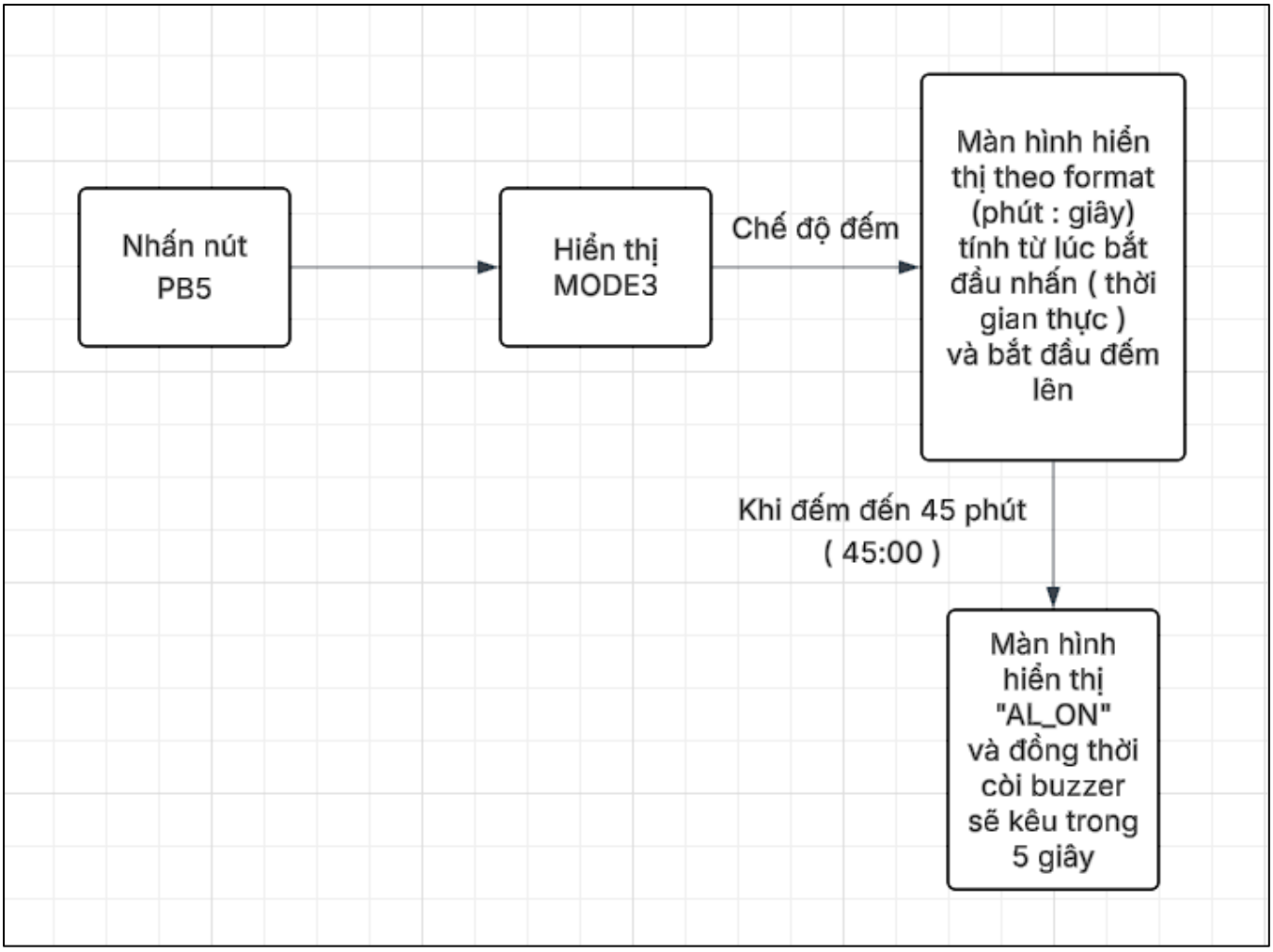
MODE1:



MODE2:



MODE3:


Nhấn nút PB5
Hiển thị MODE3
Chế độ đếm
Màn hình hiển thị theo format (phút : giây) tính từ lúc bắt đầu nhấn (thời gian thực) và bắt đầu đếm lên
Khi đếm đến 45 phút (45:00)
Màn hình hiển thị "AL_ON" và đồng thời còi buzzer sẽ kêu trong 5 giây

KẾT LUẬN

Đánh giá tổng quan:

Về sản phẩm:

Thiết bị Đồng Hồ Matrix Thời Gian Thực Đa Tính Năng là một sản phẩm tích hợp giữa phần cứng và phần mềm, sử dụng vi điều khiển STM32 làm bộ xử lý trung tâm. Thiết bị sử dụng màn hình LED ma trận 8x32 để hiển thị thời gian và các thông tin khác, đồng thời tích hợp mô-đun thời gian thực (RTC) để đảm bảo độ chính xác. Điểm nổi bật của thiết bị là có bốn chế độ hoạt động khác nhau, đáp ứng đa dạng nhu cầu sử dụng trong đời sống hằng ngày cũng như trong các ứng dụng kỹ thuật. Thiết bị này là lựa chọn lý tưởng cho các dự án học tập, nghiên cứu hoặc phát triển sản phẩm.

Về các thành viên trong nhóm:

Các thành viên nhóm 7 tích cực hỗ trợ nhau hoàn thành dự án của nhóm, với tinh thần tự giác, tự nguyện, thực hiện đề tài nghiêm túc nhằm mang lại sản phẩm hoàn chỉnh nhất.

Mặt tích cực:

- ***Tính đa năng cao:***
  Với 3 chế độ hoạt động tích hợp (hiển thị thời gian, hiển thị nhiệt độ, pomodoro timer), sản phẩm có thể đáp ứng nhiều nhu cầu sử dụng trong học tập, làm việc và sinh hoạt hàng ngày.
- ***Nền tảng phần cứng mạnh mẽ (STM32):***
  Vi điều khiển STM32 có hiệu suất xử lý cao, hỗ trợ nhiều giao tiếp ngoại vi, dễ dàng mở rộng tính năng trong tương lai như kết nối không dây, giao tiếp với cảm biến, hiển thị nâng cao...
- ***Chi phí chế tạo hợp lý:***
  Các linh kiện sử dụng phổ biến, dễ mua và có giá thành thấp. Phù hợp cho sinh viên hoặc các cá nhân làm đồ án, dự án DIY.
- ***Dễ học, dễ phát triển:***
  Là một ví dụ tiêu biểu để học lập trình nhúng, quản lý trạng thái hệ thống (state machine), và giao tiếp với các thiết bị ngoại vi như RTC, LED ma trận, nút nhấn, buzzer, …
- ***Tính tùy biến cao:***
  Có thể nâng cấp phần mềm dễ dàng, thêm chức năng mới hoặc thay đổi giao diện theo nhu cầu sử dụng.

Mặt hạn chế:

- ***Độ chính xác phụ thuộc vào chất lượng linh kiện:***

  Mặc dù là thiết bị đo thời gian thực, nhưng nếu sử dụng thạch anh kém chất lượng hoặc bị lệch nhiệt độ, sai số có thể tăng lên vài giây mỗi ngày.

- ***Tốc độ truyền dữ liệu hạn chế:***

  Các dòng phổ biến như DS1307 hay DS3231 dùng giao tiếp I2C, có tốc độ giới hạn (~100-400kHz), không phù hợp với các hệ thống yêu cầu tốc độ cao.

- **Khó tích hợp trong không gian hẹp:**

  Nếu thiết kế mạch nhỏ gọn, việc tích hợp thêm một module RTC đôi khi gây khó khăn về kích thước và bố trí linh kiện.

- Hiện tại chỉ mới phát triển được 3 tính năng, về nhiều mặt còn hạn chế về chế độ báo thức (MODE 3), cụ thể là chỉ mới thực hiện được chế độ báo thức sau 45p, trong tương lai sẽ nghiên cu phát triển thêm.

TÀI LIỆU THAM KHẢO

1. Nguồn: https://controllerstech.com/stm32/
2. Tài liệu thực hành Vi Điều Khiển bài 9: Giao tiếp I2C
3. Tài liệu lý thuyết Vi điều khiển bài tuần 7: các chuẩn giao tiếp SPI, I2C

— ***Hết***